



البرمجة بلغة ++C

جنى الخطاب

Control Structures II (Repetition)

Why Is Repetition Needed?

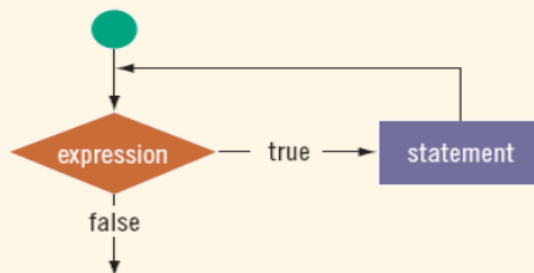
- Can input, add, and average multiple numbers using a **limited number of variables**.
يمكنك مثلا كتابة برنامج يقوم بإدخال، جمع، وحساب متوسط مجموعة من الأرقام باستخدام عدد محدود من المتغيرات.

while Looping (Repetition) Structure

```
while (expression)
    statement
```

معناها طالما ال expression evaluates to true نفذ ال statement، عندما يصبح false يتوقف عن تنفيذ ال statement.

- Statement can be simple or compound
 - Statement is called the body of the loop.
 - **while** is a reserved word.
- طبعا ممكن تكون جملة او مجموعة جمل.



- Infinite loop: continues to execute endlessly

بضل ينفذ ال statement من هون لיום القيامة.

- Avoided by including statements in loop body that assure exit condition is eventually false.

حتى نتجنب ال infinite loop نضع جمل في ال loop body تجعل ال expression يصبح false لنضمن ان ال loop سيتوقف.

هذا النوع من الجمل اسمه update statement.

EXAMPLE 5-1

Consider the following C++ program segment:

```
i = 0; //Line 1
while (i <= 20) //Line 2
{
    cout << i << " "; //Line 3
    i = i + 5; //Line 4
}
```

cout << endl;

Sample Run:

0 5 10 15 20

هذا البرنامج يطبع مضاعفات العدد 5 من الصفر للعشرين، الجملة في line 4 هي الـ update statement.

EXAMPLE 5-2

Consider the following C++ program segment:

```
i = 20; //Line 1
while (i < 20) //Line 2
{
    cout << i << " "; //Line 3
    i = i + 5; //Line 4
}
cout << endl; //Line 5
```

It is easy to overlook the difference between this example and Example 5-1. In this example, in Line 1, *i* is set to 20. Because *i* is 20, the expression *i* < 20 in the **while** statement (Line 2) evaluates to **false**. Because initially the loop entry condition, *i* < 20, is **false**, the body of the **while** loop never executes. Hence, no values are output and the value of *i* remains 20.

في هذا المثال الشرط من الأول false فما راح يدخل الـ loop ولا مرة وما راح ينفذ جملة.

Case 1: Counter-Controlled while Loops

نيجي لأول نوع من الـ loops.

- If you know exactly how many pieces of data need to be read, the while loop becomes a counter-controlled loop.

إذا بتعرف كم عدد المرات بالضبط اللي لازم يتنفذ فيها الـ loop body استخدم الـ counter-controlled loop كالآتي:

```

counter = 0;           //initialize the loop control variable
while (counter < N)    //test the loop control variable
{
    .
    .
    .
    counter++;         //update the loop control variable
    .
    .
    .
}

```

اعطي ال counter قيمة ابتدائية وفي ال loop-body كل مرة زيد قيمتها بواحد وخلي الشرط انه يوقف لما يوصل لقيمة معينة (عدد المرات اللي بدك الجمل يتم تنفيذها).
اللي بيتحكم في ال loop هو ال counter

Case 2: Sentinel-Controlled while Loops

- Sentinel variable is tested in the condition and loop ends when sentinel is encountered.

هذا النوع بيطلع من ال loop لما variable يوصل قيمة معينة ، مثلا: بدى اسمح للمستخدم ان يدخل قيم غير معروف كم عددها وعندما ينتهي من ادخال القيم أن يدخل الرقم واحد مثلا. اذا نستخدمه عند عدم معرفة عدد مرات التنفيذ.

```

cin >> variable;           //initialize the loop control variable
while (variable != sentinel) //test the loop control variable
{
    .
    .
    .
    cin >> variable;       //update the loop control variable
    .
    .
    .
}

```

هذا الكود يأخذ من المستخدم قيمة واذا هذا القيمة لا تساوي رقم معين (sentinel) رح يأخذ قيم من المستخدم ويخزنها في variable.
المتحكم في هذا ال loop هو ال sentinel القيمة اللي ما بدنا ال variable أن يساويها.

Case 3: Flag-Controlled while Loops

- A flag-controlled while loop uses a bool variable to control the loop.

في النوعين اللي قبل غالبا بيتحكم فيهم variable من نوع integer أما هذا النوع يتحكم فيه variable من نوع boolean .

- The flag-controlled while loop takes the form:

```

found = false;           //initialize the loop control variable
while (!found)           //test the loop control variable
{
    .
    .
    .
    if (expression)
        found = true; //update the loop control variable
    .
    .
    .
}

```

بيطلع من ال loop لما تتغير قيمة ال boolean variable اللي هو بتغير لما expression معين يكون true. المتحكم في هذا ال loop هو logical expression.

More on Expressions in while Statements

- The expression in a while statement can be complex
يمكن يكون ال expression مش بسيط، انه مثلا يعتمد على أكثر من شرط وأكثر من variable مثل:

```

while ( (noOfGuesses < 5) && (!isGuessed) )
{
    ...
}

```

for Looping (Repetition) Structure

مثل ال counter-controlled while loop بنستخدمه لما نكون عارفين عدد مرات التنفيذ بالضبط.

- The general form of the for statement is:

```

for (initial statement; loop condition; update statement)
    statement

```

ال initial statement عادة نضع قيمة ابتدائية ل variable يتحكم في ال loop.
ال loop condition هو الشرط اللي أول ما يصير false بيطلع من ال loop.
ال update statement حكيها عنها فوق.
مثال:

```

int counter;
for(counter = 1; counter < 10; counter++)
{
    statement;
}

```

هذا الكود بشتغل نفس الكود هذا:

```

int counter = 1;

```

```
while(counter<10)
{
statement;
counter++;
}
```

- The initial statement, loop condition, and update statement are called **for loop control statements**.

- initial statement usually initializes a variable (called the **for loop control**, or **for indexed, variable**).

-**for** is a reserved word.

طبعا في كل أنواع ال loops كل مرة يدخل فيها ال loop بشييك على ال condition.

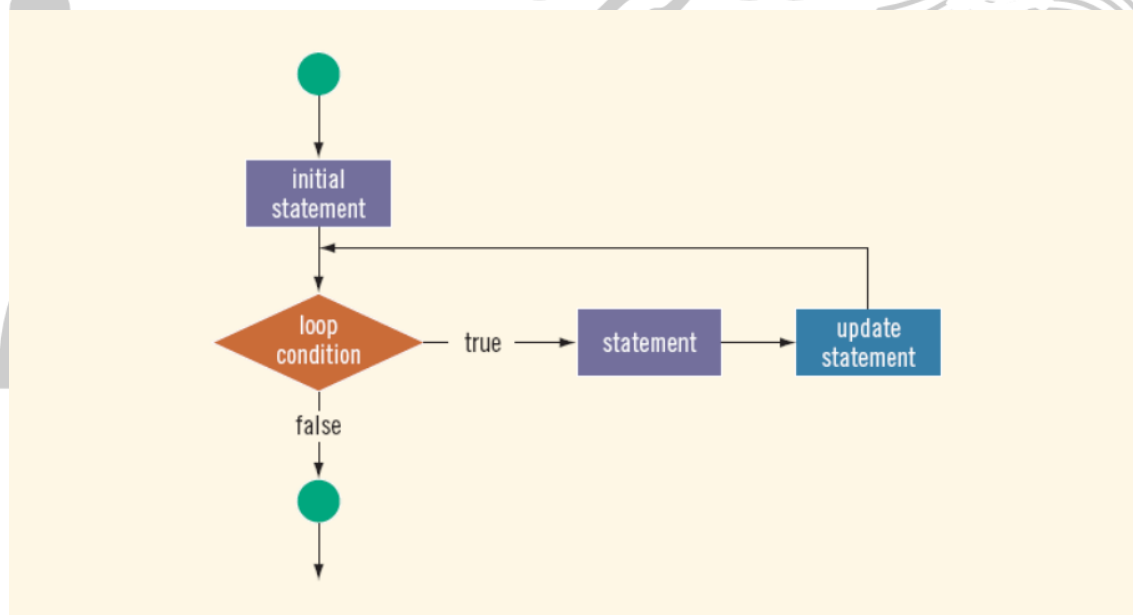


FIGURE 5-2 **for** loop

مثل ما هو موضح بالمخطط بيقرأ ال initial statement بعدين بشييك عال loop condition اذا true بنفذ ال statement بعدين بنفذ ال update statement وبرجع يشييك عال condition وهكذا.

EXAMPLE 5-7

The following **for** loop prints the first 10 non negative integers:

```
for (i = 0; i < 10; i++)  
    cout << i << " ";  
cout << endl;
```

EXAMPLE 5-8

1. The following **for** loop outputs Hello! and a star (on separate lines) five times:

```
for (i = 1; i <= 5; i++)  
{  
    cout << "Hello!" << endl;  
    cout << "*" << endl;  
}
```

2. Consider the following **for** loop:

```
for (i = 1; i <= 5; i++)  
    cout << "Hello!" << endl;  
    cout << "*" << endl;
```

في المثال الثاني الفرق بين الفرع 1 و 2 ال loop body لانه بتعرفوا اي جملة بتنتهي عند أول semicolon فإذا بدنا نضمن في ال loop أكثر من جملة نضع curly braces { }.

طبعا مسموح نستخدم variable يتحكم في ال loop يكون من نوع double بس مش رح يهمننا.

- The following is a semantic error:

EXAMPLE 5-9

The following **for** loop executes five empty statements:

```
for (i = 0; i < 5; i++);    //Line 1  
    cout << "*" << endl;    //Line 2
```

مش حكينا كل جملة بتنتهي عند أول semicolon ؟ هون ما في جمل ينفذها لما يدخل ال loop رح يعمل كل الخطوات اللي ذكرناها فوق عند المخطط بس ما رح ينفذ أي جمل!
رح يطلع من ال loop من غير ما ينفذ أي شيء ورح ينفذ الجملة اللي بعدها ويطبع نجمة.

- The following is a legal for loop:

```
for (;;)
```



```
cout << "Hello" << endl;
```

بقالك انه عادي تكتب loop مثل هيك بس هذا فعليا infinite loop لانه فعليا ما في condition لما يصير false يوقفه.

EXAMPLE 5-10

You can count backward using a **for** loop if the **for** loop control expressions are set correctly.

For example, consider the following **for** loop:

```
for (i = 10; i >= 1; i--)  
    cout << " " << i;  
cout << endl;
```

The output is:

10 9 8 7 6 5 4 3 2 1

EXAMPLE 5-11

You can increment (or decrement) the loop control variable by any fixed number. In the following **for** loop, the variable is initialized to 1; at the end of the **for** loop, *i* is incremented by 2. This **for** loop outputs the first 10 positive odd integers.

```
for (i = 1; i <= 20; i = i + 2)  
    cout << " " << i;  
cout << endl;
```

في جدول الامثلة بقالك انه ال update statement مش شرط تزيد قيمة ال variable ممكن تنقص ومش شرط مقدار الزيادة والنقصان يكون واحد حسب شو الهدف منه.

do...while Looping (Repetition) Structure

- General form of a do...while:

```
do  
    statement  
while (expression);
```

- The statement executes first, and then the expression is evaluated

في ال do..while structure بنفذ الجملة اللي بعد do بعدين بشيك عال expression عكس ال while لحاله بشيك عال condition بعدين بنفذ.

انتبهوا انه في semicolon بعد while.

- To avoid an infinite loop, body must contain a statement that makes the expression false
- The statement can be simple or compound

هدول النقاط نفس ال while loop انه لتجنب ال infinite loop يكون في update statement وانه statement اللي بعد do ممكن تكون simple أو compound.

- Loop always iterates at least once

ال statement تنفذ عالأقل مرة واحدة ليش؟ لأنه قلنا انه بنفذ ال statement بعدين بشييك عال condition فإذا كان false يكون نفذ ال statement عالأقل مرة.

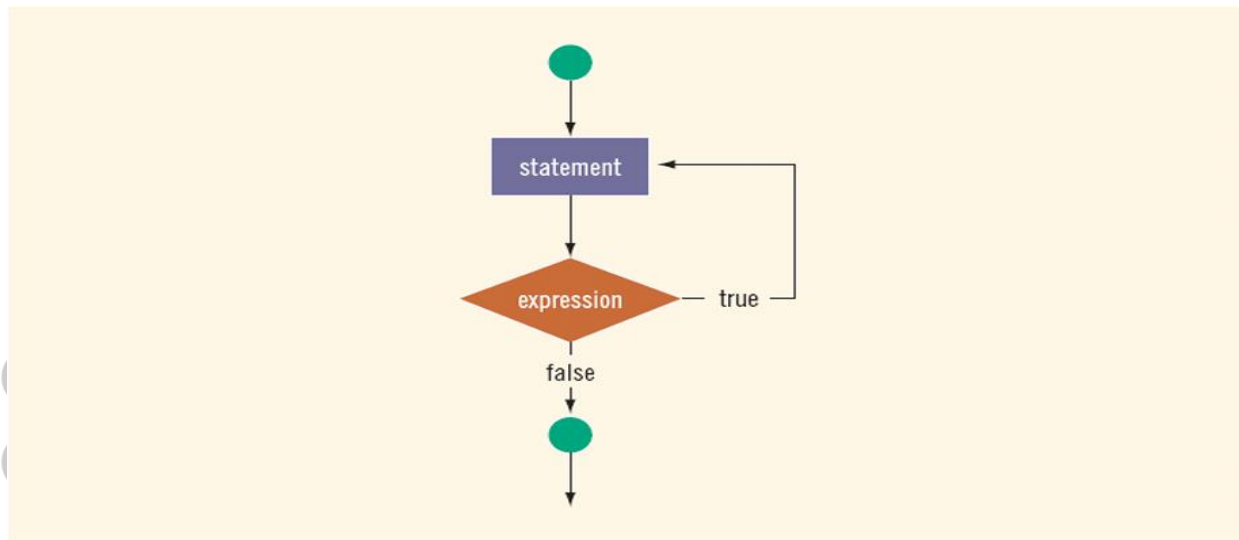


FIGURE 5-3 do...while loop

بنفذ ال statement بعدين بشييك عال expression اذا false بكمل تنفيذ الجمل اللي بعد ال loop اذا true برجع ينفذ ال statement وهكذا.

EXAMPLE 5-15

```
i = 0;

do
{
    cout << i << " ";
    i = i + 5;
}
while (i <= 20);
```

The output of this code is:

0 5 10 15 20

هذا ال program يطبع مضاعفات الخمسة من صفر لعشرين اذا بتذكروا فوق في program بعمل نفس الشيء لكن بإستخدام counter-controlled while loop.

EXAMPLE 5-16

Consider the following two loops:

```
a. i = 11;
   while (i <= 10)
   {
       cout << i << " ";
       i = i + 5;
   }
   cout << endl;

b. i = 11;
   do
   {
       cout << i << " ";
       i = i + 5;
   }
   while (i <= 10);

   cout << endl;
```

In (a), the `while` loop produces nothing. In (b), the `do...while` loop outputs the number 11 and also changes the value of `i` to 16.

المثال يوضح الفرق بين الـ `while` والـ `do...while`.

-شغلة مهمة لازم تعرفوها :

أول اشي تذكر انفقنا انه `int / int = int`

(ذكرنا سابقا انه أي عملية حسابية المعاملات فيها من نفس النوع الناتج يكون من نفس النوع برضه).

يعني $2/7$ تساوي 3 خلص النص طارت.

-تقسيم عدد على عشرة : الأثر الجميل يعمل بصمت

يحذف خانة الاحاد من الرقم مثلا: 223 تقسيم عشرة بيطلع 22 ، 867 تقسيم عشرة بيعطي 86

-حساب باقي قسمة عدد على 10:

الناتج يكون خانة الاحاد من الرقم، مثلا: 221 باقي قسمته على عشرة 1 ، 345 باقي قسمته على عشرة 5 .

-ان يكون عدد قابل للقسمة على عدد آخر يعني أ، يكون باقي قسمته على العدد الاخر صفر.

```

sum = 0;

do
{
    sum = sum + num % 10; //extract the last digit
                           //and add it to sum
    num = num / 10;       //remove the last digit
}
while (num > 0);

cout << "The sum of the digits = " << sum << endl;

if (sum % 3 == 0)
    cout << temp << " is divisible by 3" << endl;
else
    cout << temp << " is not divisible by 3" << endl;

if (sum % 9 == 0)
    cout << temp << " is divisible by 9" << endl;
else
    cout << temp << " is not divisible by 9" << endl;

```

الprogram فوق يكشف اذا الرقم يقبل القسمة على العددين 3 و 9. أول اشي كيف بنعرف انه الرقم بقسم على 3 و 9 أو لا؟ في الرياضيات اذا كان مجموع خانات العدد يقبل القسمة على 3 أو 9 يكون العدد قابل للقسمة على 3 أو 9 ، مثلا العدد 12 قابل للقسمة على 3 لانه مجموع خاناته 2+1 يقبل القسمة على 3 (هاي الامور اللي ممكن تكون مش كثير معروفة غالبا تعطى مع السؤال).

Choosing the Right Looping Structure

كيف نختار ال looping structure الصح (for, while, do...while)

-If you know or can determine in advance the number of repetitions needed, the forloop is the correct choice

اذا بتعرف عدد التكرار بالضبط استخدم for.

-If you do not know and cannot determine in advance the number of repetitions needed, and it could be zero, use a while loop.

اذا ما بتعرف عدد التكرار وبتعرف انه ممكن يكون صفر استخدم while .

-If you do not know and cannot determine in advance the number of repetitions needed, and it is at least one, use a do...whileloop

اذا ما بتعرف عدد التكرار وبتعرف انه لازم عالأقل يكون مرة استخدم do...while.

break and continue Statements

جمل break و continue يتحكموا في استمرارية أو توقف ال loop.

- break statement is used for two purposes:
 - To exit early from a loop
- Can eliminate the use of certain (flag) variables
 - To skip the remainder of the switch structure

ال break statement شفناها في switch structure لنعمل skip لباقي ال cases. ممكن نستخدمها لنطلع من ال loop عند شرط معين (هيك بنستعيز عن ال flag) كالاتي:

```
bool flag = true;
while(flag)
{
    ....
    if(expression)
        flag= false;
}
```

بدل ما نستخدم flag نستخدم break:

```
while(expression)
{
    ....
    if(expression)
        break;
}
```

لما الكومبايلر يقرأ break بيطلع من ال loop وينفذ أ، ل جملة بعد ال loop.

- continue is used in while, for, and do...while structures.

-It skips remaining statements and proceeds with the next iteration of the loop
بتعمل skip للجمل الباقية وبتكمل في ال loop.
مثال:

```
for(i=1;i<=6;i++)
{
    if (i == 5)
        continue;
```

```
cout<<i<<" ";  
}
```

the output:

1 2 3 4 6

هذا ال program يطبع الارقام من 1 الى 6 بس بعمل skip عن رقم خمسة.

Nested Control Structures

يعني loop داخله loop ثاني.

- To create the following pattern:

```
*  
**  
***  
****  
*****
```

- We can use the following code:

```
for(i = 1; i <= 5 ; i++)  
{  
    for(j = 1; j <= i; j++)  
        cout << "*";  
    cout << endl;  
}
```

- What is the result if we replace the first for statement with the following? for(i = 5; i >= 1; i--)

- Answer:

```
*****  
****  
***  
**  
*
```

Problem-Solving Part

1.write a C++ program that prompt the user to input positive integer then print the sum of its digits.

2. Write a C++ program that continuously prompts the user to enter numbers. The program should:

1. Print each entered number back to the user.
2. Stop asking for numbers when the user enters a negative number.

3. Write a C++ program that allows a user to enter a series of integers. The program should continue prompting the user until they decide to stop by entering **-1**. Once the user finishes entering numbers, the program should search for a specific target number within the entered integers. If the target number is found, the program should print a message and stop further searching.

4. Write a C++ program that reads a series of integers from the user and calculates the sum of all positive integers entered. The program should skip any negative numbers entered

```
#include <iostream>
using namespace std;

int main() {
    int number;
    int sum = 0;

    cout << "Enter a positive integer: "; // Prompt the user to input a positive integer
    cin >> number;

    // Calculate the sum of digits
    do
    {
        sum+=number%10;
        number/=10;
    }
    while(number!=0);
    cout << "The sum of digits is: "<< sum << endl;

    return 0;
}
```

```

#include <iostream>
using namespace std;

int main() {
    bool flag = true; // Flag to control the loop
    int number;

    cout << "Enter numbers to add to the sum. Enter a negative number to stop." << endl;

    while (flag) {
        cin >> number;

        if (number < 0)
            flag = false; // Set the flag to false to exit the loop
        else
            cout << "You entered: " << number << endl;
    }

    return 0;
}

```

3.

```

int main() {
    const int target = 3; // Target number to search for
    int number;
    bool found = false; // Flag to indicate if target is found

    cout << "Enter an integer (enter -1 to stop): ";
    cin >> number;

    while (number != -1) {
        if (number == target) {
            found = true;
            break; // Exit the loop immediately if target is found
        }
        cout << "Enter an integer (enter -1 to stop): ";
        cin >> number;
    }

    if (found) {
        cout << "Target number " << target << " is found in the list." << endl;
    } else {
        cout << "Target number " << target << " is not found." << endl;
    }

    return 0;
}

```

4.


```
int main() {  
    int number;  
    int sum = 0; // Variable to store the sum of positive integers  
  
    cout << "Enter an integer (enter -1 to stop): ";  
    cin >> number;  
  
    while (number != -1) {  
        if (number < 0) {  
            // Skip negative numbers  
            cout << "Negative number entered. Skipping..." << endl;  
            cout << "Enter an integer (enter -1 to stop): ";  
            cin >> number;  
            continue; // Skip to the next iteration  
        }  
  
        // Add positive number to sum  
        sum += number;  
  
        cout << "Enter an integer (enter -1 to stop): ";  
        cin >> number;  
    }  
  
    cout << "The sum of positive integers entered is: " << sum << endl;  
    return 0;  
}
```

احسن

الأثر الجميل يعمل بصمت